

```

from sklearn.model_selection import train_test_split
import api_interface
import train
import trainingparams
import serializer
from dataset import Dataset

class Training:
    def __init__(self, dataset: Dataset = None):
        self.dataset = dataset
        self.test_dataset = None
        self.training = None
        self.crossvalidation = None
        self.algorithms = None
        self.target = None
        self.features = None

    def set_dataset(self, dataset: Dataset):
        self.dataset = dataset

    def train_and_evaluate(self):
        dataset = self.dataset.as_dataframe()

        # Separar las características (X) de la variable objetivo (y)
        X = dataset.drop(columns=self.target)
        y = dataset[self.target]

        ##### ENTRENAMIENTO CUSTOM #####
        # Si el usuario especifica las columnas, usarlas
        if self.features and self.features != "null":
            X = X[self.features]

        # Convertir a valores numpy
        X = X.values
        y = y.values

        # Dividir los datos en entrenamiento y prueba
        X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=(100 -
self.crossvalidation) / 100, random_state=42)

        # Entrenar los modelos
        trained_models = train.train_models(X_train, y_train, self.algorithms)

        # Evaluar los modelos
        evaluation_results = {}
        for model in trained_models:
            if isinstance(model, tuple(serializer.classification_models)):
                eval_results = train.evaluate_classification_models([model], X_test, y_test)
            else:
                eval_results = train.evaluate_regression_models([model], X_test, y_test)
            evaluation_results.update(eval_results)

        ##### ENTRENAMIENTO AUTOMÁTICO #####
        if (self.recommendations == true)
            # Entrenamiento automático detectando columnas más relevantes
            self.train_with_important_features()

            # Entrenar modelos con parámetros recomendados
            recommended_params = trainingparams.train_recommendedparams(X_train, y_train,
self.algorithms)

        ##### ENVÍO DE DATOS #####
        # Enviar resultados de evaluación y parámetros recomendados a la API
        api_interface.POST_modelevel(evaluation_results) #,recommended_params)

    def train_with_important_features(self, k=10, ):
        dataset = self.dataset.as_dataframe()

```

```

# Separar las características (X) de la variable objetivo (y)
X = dataset.drop(columns=self.target).values
y = dataset[self.target].values

# Seleccionar las características más relevantes
X_new, selected_features = train.select_features(X, y, task_type=task_type, k=k)
print(f"Selected features: {dataset.columns[selected_features]}")

# Dividir los datos en entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(X_new, y, test_size=(100 -
self.crossvalidation) / 100, random_state=42)

# Entrenar los modelos
trained_models = train.train_models(X_train, y_train, self.algorithms)

# Evaluar los modelos
evaluation_results = {}
for model in trained_models:
    if task_type == 'classification':
        eval_results = train.evaluate_classification_models([model], X_test, y_test)
    else:
        eval_results = train.evaluate_regression_models([model], X_test, y_test)
    evaluation_results.update(eval_results)

# Enviar resultados de evaluación a la API
api_interface.POST_modelevel(evaluation_results)

def predict_and_evaluate(self, model_path, X):
    # Cargar el modelo desde el archivo
    model = serializer.from_pickle(model_path)
    y_pred = model.predict(X)
    return y_pred

```